

华东师范大学数据科学与工程学院实验报告

课程名称: AI 基础	年级: 2024	上机实践时间: 2026.04.09
指导教师: 杨彬	姓名: 孙育泉	
上机实践名称: Maze	学号: 10234900421	

I 实验任务

给定一个 $n \times m$ 的二维整数数组，用来表示一个迷宫，数组中只包含 0 或 1，其中 0 表示可以走的路，1 表示不可通过的墙壁。最初，有个人位于左上角 $(1, 1)$ 处，已知该人每次可以向上、下、左、右任意一个方向移动一个位置。

请问，该人从左上角移动至右下角 (n, m) 处最少需要移动多少次。数据保证 $(1, 1)$ 与 (n, m) 处的值均为 0，且至少存在一条路径可以到达。并且需要进行可视化。

考虑使用这些算法：DFS, BFS, Dijkstra, A*。

II 使用环境

编程语言：Python 3

III 实验过程

III.1 基础封装



Idea

为了代码的可维护性与解藕性，这里对代码进行了很大的改动，进行了模块化。

首先，为了进行后续的可视化与维护每次探索的状态，我们将迷宫分装成了一个 `Maze` 类，并且定义了一个 `SolverState` 数据类来保存每一步的快照，方便后续的可视化展示。

py

```
1 # maze.py
2 from dataclasses import dataclass
3 from typing import List, Tuple, Set
4
5
6 @dataclass
7 class SolverState:
8     """用于寻路算法与可视化解耦的数据模型，保存每一步的快照"""
9
10     current: Tuple[int, int]
11     explored: List[Tuple[int, int]]
12     frontier: List[Tuple[int, int]] # 待探索队列（栈/队列/优先队列）
13     path: List[Tuple[int, int]]
14
15
16 class Maze:
17     """迷宫的基础数据结构"""
18
19     def __init__(
```

```

20         self,
21         grid: List[List[int]],
22         start: Tuple[int, int] = (0, 0),
23         goal: Tuple[int, int] = None,
24     ):
25         self.grid = grid
26         self.rows = len(grid)
27         self.cols = len(grid[0])
28         self.start = start
29         self.goal = goal if goal else (self.rows - 1, self.cols - 1)
30
31     def is_wall(self, r: int, c: int) → bool:
32         return self.grid[r][c] == 1
33
34     def is_valid_move(self, r: int, c: int) → bool:
35         return 0 ≤ r < self.rows and 0 ≤ c < self.cols and not
            self.is_wall(r, c)
36
37     def get_neighbors(self, r: int, c: int) → List[Tuple[int, int]]:
38         directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
39         neighbors = []
40         for dr, dc in directions:
41             nr, nc = r + dr, c + dc
42             if self.is_valid_move(nr, nc):
43                 neighbors.append((nr, nc))
44         return neighbors
45

```

III.2 迷宫生成

然后，我们需要生成迷宫，用于探究算法在不同的迷宫类型下的性能表现。为此，我们定义了一个 `MazeGenerator` 类，包含多种不同拓扑结构的迷宫生成策略。每种生成方法都保证生成的迷宫至少存在一条解，并且返回一个 `Maze` 类的实例，方便后续的寻路算法使用。

py

```

1 # generators.py
2 import random
3 from collections import deque
4 from maze import Maze
5
6
7 class MazeGenerator:
8     """
9     迷宫生成器：包含多种不同拓扑结构的迷宫生成策略。
10    所有生成方法均返回 maze.Maze 类的实例。
11    """
12
13    @staticmethod
14    def _is_solvable(grid: list[list[int]]) → bool:
15        """内部辅助函数：使用轻量级 BFS 快速验证网络是否至少存在一条解"""
16        ...
17
18    ...

```

接下来，介绍一下这几种迷宫生成方法：

(1) `generate_random_density`: 生成**随机密度迷宫**, 墙壁的分布完全随机, 但保证至少存在一条解.

- **原理**: 算法遍历网格中的每一个格子, 并生成一个 0 到 1 之间的随机数. 如果该数字小于设定的阈值, 则将该位置设为障碍物, 否则设为平地. 由于纯随机可能导致起点和终点被彻底封死, 每次生成后使用轻量级 BFS 快速跑一遍, 如果无解就直接丢弃并重新生成, 直到刷出一张有解的地图.
- **地图特点**: 毫无章法、高度碎片化. 它不存在传统意义上的“走廊”或“死胡同”, 而是布满了大小不一的开阔空地和随机散落的像素块.
- **接口**:

py

```
1 @staticmethod
2 def generate_random_density(
3     rows: int, cols: int, wall_probability: float = 0.25
4 ) → Maze:
5     """生成随机密度的迷宫 (保证有解) """
6     ...
```

(2) `generate_perfect_maze`: 生成**完美迷宫**, 即没有环路的迷宫, 所有死胡同都只有一个出口.

- **原理**: 算法模拟了一个“不撞南墙不回头”的矿工. 从起点开始, 随机选择一个相邻且未被打通的格子, 挖穿中间的墙, 并把新位置压入栈中. 如果走到一个周围所有格子都已经被挖过的地方 (死胡同), 就从栈中弹出上一个位置进行“回溯”, 直到找到新的未开发分支, 继续往下挖.
- **地图特点**: 高曲折度, 长且深的死胡同. 它拥有大量的分叉和转弯, 使得路径错综复杂, 但每条死胡同都只能通向一个出口.
- **接口**:

py

```
1 @staticmethod
2 def generate_perfect_maze(rows: int, cols: int) → Maze:
3     """使用 Randomized DFS 生成完美迷宫 (高曲折度, 长且深的死胡同) """
4     ...
```

(3) `generate_braid_maze`: 生成**编织迷宫**, 即多解迷宫, 存在环路.

- **原理**: 它是在完美迷宫的基础上进行二次加工的产物. 算法遍历已经生成好的完美迷宫, 找出所有三面环墙的“死胡同”. 然后, 根据设定的概率, 随机打通死胡同的一堵墙, 使其与相邻的另一条道路强行连通. 这样就形成了环路, 增加了多解的可能性.
- **地图特点**: 迷宫中充满了“环路”, 基本上消灭了死胡同. 这意味着从起点到终点存在无数种不同的有效路径, 长短不一.
- **接口**:

py

```
1 @staticmethod
2 def generate_braid_maze(
3     rows: int, cols: int, braid_probability: float = 0.5
4 ) → Maze:
5     """生成编织迷宫 (多解迷宫, 存在环路) """
6     ...
```

(4) `generate_cave_maze`: 生成**洞穴迷宫**, 即类似自然洞穴的迷宫.

- **原理**: 算法基于细胞自动机的思想. 首先随机生成一个初始状态的网格, 其中每个格子以一定概率被设为墙壁. 然后, 算法迭代地更新网格状态: 对于每个格子, 统计其周围 8

个邻居中有多少是墙壁.如果邻居中墙壁数量超过某个阈值,则该格子在下一轮迭代中变成墙壁;反之则变成平地.经过多次迭代后,网格会逐渐演化成类似自然洞穴的结构.

- **地图特点:** 拥有大量不规则的开阔空间和狭窄通道,整体感觉更像是自然形成的洞穴系统,而不是人工设计的迷宫.
- **接口:**

py

```
1 @staticmethod
2 def generate_cave_maze(
3     rows: int, cols: int, initial_wall_prob: float = 0.45, iterations:
4     int = 4
5 ) → Maze:
6     """利用细胞自动机生成类似自然洞穴的迷宫"""
7     ...
```

- (5) `generate_recursive_division`: 生成**递归分割迷宫**,即拥有长直走廊和整齐的矩形区块的迷宫.

- **原理:** 算法采用分治的思想.首先将整个网格视为一个大矩形区域,在其中随机选择一个位置竖直或水平划分成两个子区域,并在划分线上随机打通一个洞作为通路.然后对每个子区域递归地执行同样的划分过程,直到子区域小到无法再划分为止.最终形成的迷宫由许多长直的走廊和整齐的矩形区块组成.
- **地图特点:** 拥有大量长直走廊和规整的矩形区块.它的结构相对简单,路径较为直接,但仍然存在一些死胡同和分叉.
- **接口:**

py

```
1 @staticmethod
2 def generate_recursive_division(rows: int, cols: int) → Maze:
3     """使用递归分割法生成迷宫 (拥有长直走廊和整齐的矩形区块) """
4     ...
```

- (6) `generate_prim_maze`: 生成**Prim 迷宫**,即极多超短死胡同,强烈的中心辐射感的迷宫.

- **原理:** 算法从起点开始,将其加入一个“已访问”集合.然后,算法将起点的所有邻居加入一个优先队列中,优先级根据某种随机权重来确定.每次从优先队列中取出优先级最高的邻居,如果该邻居未被访问过,则将其加入“已访问”集合,并打通它与它的父节点之间的墙.然后,将该邻居的所有未访问过的邻居加入优先队列中.这个过程持续进行,直到优先队列为空为止.
- **地图特点:** 极多超短死胡同,强烈的中心辐射感.它的路径结构非常独特,中心区域通常会有更多的分叉和死胡同,而外围则相对简单.
- **接口:**

py

```
1 @staticmethod
2 def generate_prim_maze(rows: int, cols: int) → Maze:
3     """使用随机 Prim 算法生成迷宫 (极多超短死胡同,强烈的中心辐射感) """
4     ...
```



Idea

这里放上 grid 的可视化图,展示一下不同生成方法生成的迷宫长什么样子.

III.3 搜索算法

在生成了不同类型的迷宫之后，我们需要实现多种搜索算法来寻找从起点到终点的最短路径。为此，我们定义了一个 `MazeSolver` 类，包含 DFS、BFS、Dijkstra 和 A* 四种算法的实现。每个算法都接受一个 `Maze` 实例作为输入，并返回一个 `SolverState` 的迭代器，方便后续的可视化展示。

(1) DFS：深度优先搜索，维护一个栈，每次递归调用直到找到终点或无路可走时回退。

py

```

1 @staticmethod
2 def dfs(maze: Maze) → Iterator[SolverState]:
3     stack = [(maze.start, [maze.start])]
4     visited = set()
5     explored_order = []
6
7     while stack:
8         current, path = stack.pop()
9         if current in visited:
10             continue
11
12         visited.add(current)
13         explored_order.append(current)
14
15         # 提取栈中等待探索的节点
16         frontier = [node for (node, p) in stack if node not in visited]
17         yield SolverState(current, list(explored_order), frontier, path)
18
19         if current == maze.goal:
20             return
21
22         for nr, nc in maze.get_neighbors(*current):
23             if (nr, nc) not in visited:
24                 stack.append(((nr, nc), path + [(nr, nc)]))

```

(2) BFS：广度优先搜索，维护一个队列，每次探索当前层的所有节点后再进入下一层。

py

```

1 @staticmethod
2 def bfs(maze: Maze) → Iterator[SolverState]:
3     queue = deque([(maze.start, [maze.start])])
4     visited = {maze.start}
5     explored_order = []
6
7     while queue:
8         current, path = queue.popleft()
9         explored_order.append(current)
10
11         frontier = [node for (node, p) in queue]
12         yield SolverState(current, list(explored_order), frontier, path)
13
14         if current == maze.goal:
15             return
16
17         for nr, nc in maze.get_neighbors(*current):
18             if (nr, nc) not in visited:
19                 visited.add((nr, nc))
20                 queue.append(((nr, nc), path + [(nr, nc)]))

```

(3) Dijkstra：维护一个优先队列，每次选择当前路径代价最小的节点进行扩展。

py

```

1 @staticmethod
2 def dijkstra(maze: Maze) → Iterator[SolverState]:
3     # 优先队列存储: (当前累计代价, 当前坐标, 到达当前坐标的路径)
4     pq = [(0, maze.start, [maze.start])]
5     # 记录到达每个节点的最短已知代价
6     costs = {maze.start: 0}
7
8     explored_order = []
9     explored_set = set()
10
11     while pq:
12         current_cost, current, path = heapq.heappop(pq)
13
14         # 如果该节点已经被完全探索过, 说明队列里存放的是冗余的历史较高代价记录, 直接跳过
15         if current in explored_set:
16             continue
17
18         explored_set.add(current)
19         explored_order.append(current)
20
21         # 提取优先队列中还在等待探索的节点 (过滤掉已经处理完的)
22         frontier = list({node for cost, node, p in pq if node not in explored_set})
23
24         # yield 当前状态给可视化器
25         yield SolverState(current, list(explored_order), frontier, path)
26
27         if current == maze.goal:
28             return
29
30         for nr, nc in maze.get_neighbors(*current):
31             new_cost = current_cost + 1 # 迷宫中每走一步的代价视为 1
32
33             # 只有找到更优路径时, 才更新代价并加入队列
34             if new_cost < costs.get((nr, nc), float("inf")):
35                 costs[(nr, nc)] = new_cost
36                 heapq.heappush(pq, (new_cost, (nr, nc), path + [(nr, nc)]))

```

(4) A*: 在 Dijkstra 的基础上加入启发式函数, 优先探索估价函数值最小的节点.

py

```

1 @staticmethod
2 def a_star(maze: Maze) → Iterator[SolverState]:
3     def manhattan(p1, p2):
4         return abs(p1[0] - p2[0]) + abs(p1[1] - p2[1])
5
6     g_score = {maze.start: 0}
7     pq = [(manhattan(maze.start, maze.goal), maze.start, [maze.start])]
8     explored_order = []
9     explored_set = set()
10
11     while pq:
12         _, current, path = heapq.heappop(pq)
13
14         if current in explored_set:

```

```

15         continue
16
17     explored_set.add(current)
18     explored_order.append(current)
19     frontier = list({node for f, node, p in pq if node not in
20                     explored_set})
21
22     yield SolverState(current, list(explored_order), frontier, path)
23
24     if current == maze.goal:
25         return
26
27     for nr, nc in maze.get_neighbors(*current):
28         tentative_g = g_score.get(current, float("inf")) + 1
29         if tentative_g < g_score.get((nr, nc), float("inf")):
30             g_score[(nr, nc)] = tentative_g
31             f_score = tentative_g + manhattan((nr, nc), maze.goal)
32             heapq.heappush(pq, (f_score, (nr, nc), path + [(nr, nc)]))

```



Idea

实际上, 由于在这个迷宫问题中, 所有边的边权均为 1, 因此 Dijkstra 的优先队列本质上和 BFS 的普通队列是等价的. 因此在这个特定问题中, Dijkstra 和 BFS 的唯一的区别是 Dijkstra 维护了一个额外的成本字典来记录到达每个节点的最短已知代价, 而 BFS 则只需要维护一个简单的访问集合.

III.4 可视化渲染

为了更直观地展示搜索算法的执行过程, 我们实现了一个 MazeVisualizer 类, 使用 Matplotlib 来动态渲染迷宫的状态. 每当搜索算法 yield 一个新的 SolverState 时, MazeVisualizer 就会更新图像, 显示当前节点、已探索的节点、待探索的节点以及当前路径.

py

```

1 class MazeVisualizer:
2     def __init__(self, maze: Maze):
3         self.maze = maze
4         self.fig, self.ax = plt.subplots(figsize=(10, 8))
5         self._setup_plot()
6
7         # 保存动态绘制对象的引用以便清除
8         self.dynamic_patches = {
9             "explored": [],
10            "frontier": [],
11            "current": [],
12            "goal": [],
13            "path": [],
14        }
15
16    def _setup_plot(self):
17        ...
18
19    def _clear_patches(self, key: str):
20        while self.dynamic_patches[key]:
21            self.dynamic_patches[key].pop().remove()
22

```

```
23     def _add_rect(self, r, c, color, alpha=1.0) → patches.Rectangle:
24         rect = patches.Rectangle(
25             (c - 0.5, r - 0.5), 1, 1, linewidth=0, facecolor=color, alpha=alpha
26         )
27         return self.ax.add_patch(rect)
28
29     def _update_frame(self, state: SolverState):
30         ...
31
32     def animate(
33         self, solver_generator: Iterator[SolverState], interval=100,
34         save_path=None
35     ):
36         ...
```

IV 总结

i Info

完整代码见压缩包其余代码文件.